

POSIX Threads Programming

Author: Blaise Barney, Lawrence Livermore National Laboratory

UCRL-MI-133316

Table of Contents

1. [Abstract](#)
2. [Pthreads Overview](#)
 1. [What is a Thread?](#)
 2. [What are Pthreads?](#)
 3. [Why Pthreads?](#)
 4. [Designing Threaded Programs](#)
3. [The Pthreads API](#)
4. [Compiling Threaded Programs](#)
5. [Thread Management](#)
 1. [Creating and Terminating Threads](#)
 2. [Passing Arguments to Threads](#)
 3. [Joining and Detaching Threads](#)
 4. [Stack Management](#)
 5. [Miscellaneous Routines](#)
6. [Exercise 1](#)
7. [Mutex Variables](#)
 1. [Mutex Variables Overview](#)
 2. [Creating and Destroying Mutexes](#)
 3. [Locking and Unlocking Mutexes](#)
8. [Condition Variables](#)
 1. [Condition Variables Overview](#)
 2. [Creating and Destroying Condition Variables](#)
 3. [Waiting and Signaling on Condition Variables](#)
9. [Monitoring, Debugging and Performance Analysis Tools for Pthreads](#)
10. [LLNL Specific Information and Recommendations](#)
11. [Topics Not Covered](#)
12. [Exercise 2](#)
13. [References and More Information](#)
14. [Appendix A: Pthread Library Routines Reference](#)

Abstract

In shared memory multiprocessor architectures, threads can be used to implement parallelism. Historically, hardware vendors have implemented their own proprietary versions of threads, making portability a concern for software developers. For UNIX systems, a standardized C language threads programming interface has been specified by the IEEE POSIX 1003.1c standard. Implementations that adhere to this standard are referred to as POSIX threads, or Pthreads.

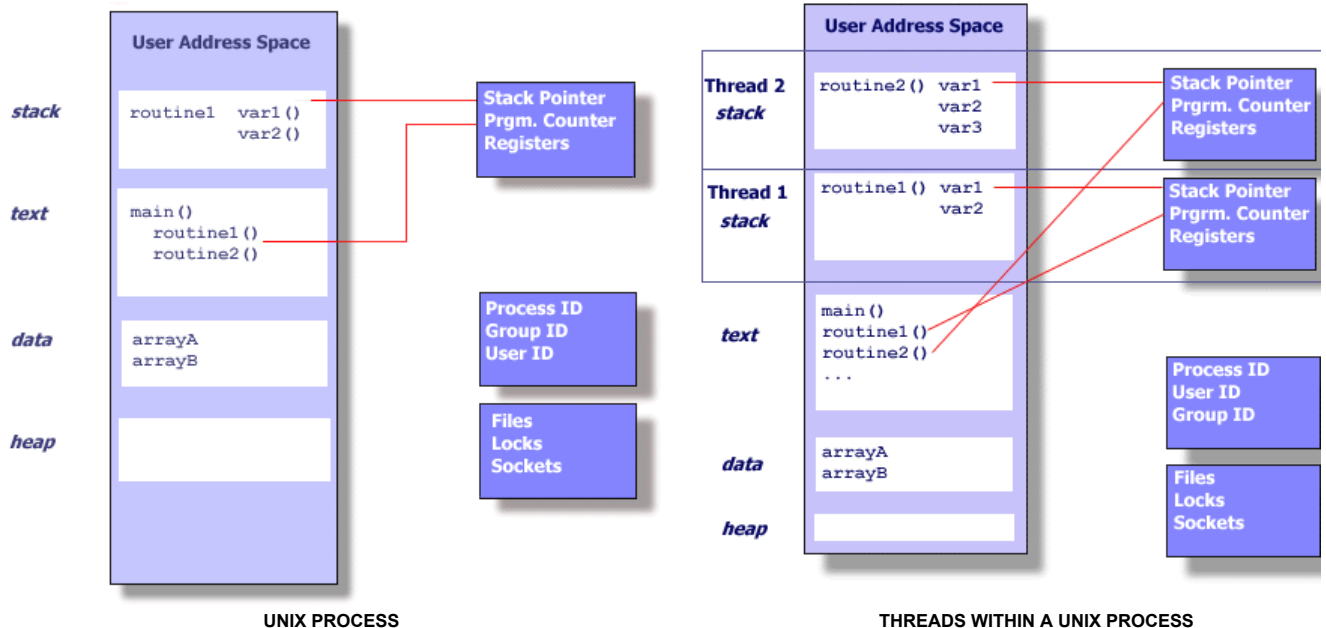
The tutorial begins with an introduction to concepts, motivations, and design considerations for using Pthreads. Each of the three major classes of routines in the Pthreads API are then covered: Thread Management, Mutex Variables, and Condition Variables. Example codes are used throughout to demonstrate how to use most of the Pthreads routines needed by a new Pthreads programmer. The tutorial concludes with a discussion of LLNL specifics and how to mix MPI with pthreads. A lab exercise, with numerous example codes (C Language) is also included.

Level/Prerequisites: This tutorial is ideal for those who are new to parallel programming with pthreads. A basic understanding of parallel programming in C is required. For those who are unfamiliar with Parallel Programming in general, the material covered in [EC3500: Introduction to Parallel Computing](#) would be helpful.

Pthreads Overview

What is a Thread?

- Technically, a thread is defined as an independent stream of instructions that can be scheduled to run as such by the operating system. But what does this mean?
- To the software developer, the concept of a "procedure" that runs independently from its main program may best describe a thread.
- To go one step further, imagine a main program (a.out) that contains a number of procedures. Then imagine all of these procedures being able to be scheduled to run simultaneously and/or independently by the operating system. That would describe a "multi-threaded" program.
- How is this accomplished?
- Before understanding a thread, one first needs to understand a UNIX process. A process is created by the operating system, and requires a fair amount of "overhead". Processes contain information about program resources and program execution state, including:
 - Process ID, process group ID, user ID, and group ID
 - Environment
 - Working directory.
 - Program instructions
 - Registers
 - Stack
 - Heap
 - File descriptors
 - Signal actions
 - Shared libraries
 - Inter-process communication tools (such as message queues, pipes, semaphores, or shared memory).



- Threads use and exist within these process resources, yet are able to be scheduled by the operating system and run as independent entities largely because they duplicate only the bare essential resources that enable them to exist as executable code.
- This independent flow of control is accomplished because a thread maintains its own:
 - Stack pointer
 - Registers
 - Scheduling properties (such as policy or priority)
 - Set of pending and blocked signals
 - Thread specific data.
- So, in summary, in the UNIX environment a thread:
 - Exists within a process and uses the process resources
 - Has its own independent flow of control as long as its parent process exists and the OS supports it
 - Duplicates only the essential resources it needs to be independently schedulable
 - May share the process resources with other threads that act equally independently (and dependently)
 - Dies if the parent process dies - or something similar
 - Is "lightweight" because most of the overhead has already been accomplished through the creation of its process.
- Because threads within the same process share resources:
 - Changes made by one thread to shared system resources (such as closing a file) will be seen by all other threads.
 - Two pointers having the same value point to the same data.
 - Reading and writing to the same memory locations is possible, and therefore requires explicit synchronization by the programmer.

Pthreads Overview

What are Pthreads?

- Historically, hardware vendors have implemented their own proprietary versions of threads. These implementations differed substantially from each other making it difficult for programmers to develop portable threaded applications.
- In order to take full advantage of the capabilities provided by threads, a standardized programming interface was required.
 - For UNIX systems, this interface has been specified by the IEEE POSIX 1003.1c standard (1995).
 - Implementations adhering to this standard are referred to as POSIX threads, or Pthreads.
 - Most hardware vendors now offer Pthreads in addition to their proprietary API's.
- The POSIX standard has continued to evolve and undergo revisions, including the Pthreads specification.
- Some useful links:
 - standards.ieee.org/findstds/standard/1003.1-2008.html
 - www.opengroup.org/austin/papers/posix_faq.html
 - www.unix.org/version3/ieee_std.html
- Pthreads are defined as a set of C language programming types and procedure calls, implemented with a `pthread.h` header/include file and a thread library - though this library may be part of another library, such as `libc`, in some implementations.

Pthreads Overview

Why Pthreads?

► Light Weight:

- When compared to the cost of creating and managing a process, a thread can be created with much less operating system overhead. Managing threads requires fewer system resources than managing processes.
- For example, the following table compares timing results for the `fork()` subroutine and the `pthread_create()` subroutine. Timings reflect 50,000 process/thread creations, were performed with the `time` utility, and units are in seconds, no optimization flags.

Note: don't expect the system and user times to add up to real time, because these are SMP systems with multiple CPUs/cores working on the problem at the same time. At best, these are approximations run on local machines, past and present.

Platform	fork ()			pthread_create ()		
	real	user	sys	real	user	sys
Intel 2.6 GHz Xeon E5-2670 (16 cores/node)	8.1	0.1	2.9	0.9	0.2	0.3
Intel 2.8 GHz Xeon 5660 (12 cores/node)	4.4	0.4	4.3	0.7	0.2	0.5
AMD 2.3 GHz Opteron (16 cores/node)	12.5	1.0	12.5	1.2	0.2	1.3
AMD 2.4 GHz Opteron (8 cores/node)	17.6	2.2	15.7	1.4	0.3	1.3
IBM 4.0 GHz POWER6 (8 cpus/node)	9.5	0.6	8.8	1.6	0.1	0.4
IBM 1.9 GHz POWER5 p5-575 (8 cpus/node)	64.2	30.7	27.6	1.7	0.6	1.1
IBM 1.5 GHz POWER4 (8 cpus/node)	104.5	48.6	47.2	2.1	1.0	1.5
INTEL 2.4 GHz Xeon (2 cpus/node)	54.9	1.5	20.8	1.6	0.7	0.9
INTEL 1.4 GHz Itanium2 (4 cpus/node)	54.5	1.1	22.2	2.0	1.2	0.6

[Source](#) [fork vs thread.txt](#)

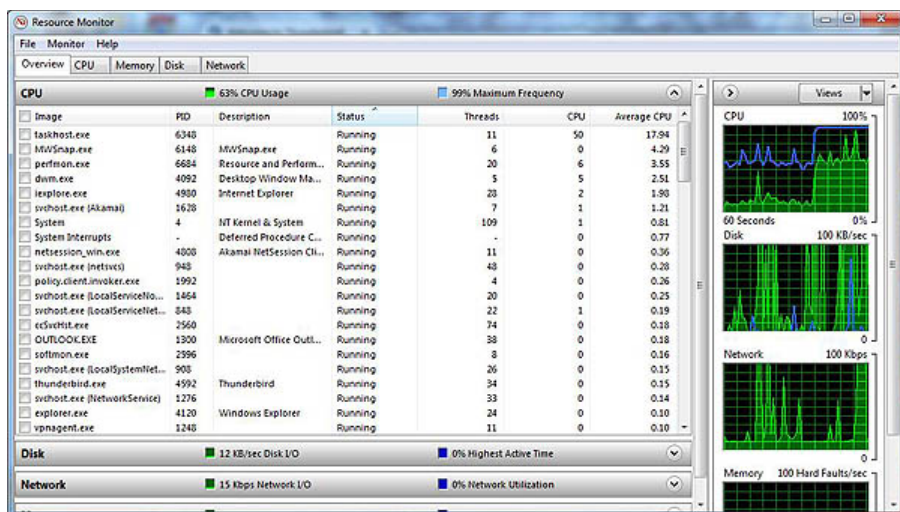
Efficient Communications/Data Exchange:

- The primary motivation for considering the use of Pthreads in a high performance computing environment is to achieve optimum performance. In particular, if an application is using MPI for on-node communications, there is a potential that performance could be improved by using Pthreads instead.
- MPI libraries usually implement on-node task communication via shared memory, which involves at least one memory copy operation (process to process).
- For Pthreads there is no intermediate memory copy required because threads share the same address space within a single process. There is no data transfer, per se. It can be as efficient as simply passing a pointer.
- In the worst case scenario, Pthread communications become more of a cache-to-CPU or memory-to-CPU bandwidth issue. These speeds are much higher than MPI shared memory communications.
- For example: some local comparisons, past and present, are shown below:

Platform	MPI Shared Memory Bandwidth (GB/sec)	Pthreads Worst Case Memory-to-CPU Bandwidth (GB/sec)
Intel 2.6 GHz Xeon E5-2670	4.5	51.2
Intel 2.8 GHz Xeon 5660	5.6	32
AMD 2.3 GHz Opteron	1.8	5.3
AMD 2.4 GHz Opteron	1.2	5.3
IBM 1.9 GHz POWER5 p5-575	4.1	16
IBM 1.5 GHz POWER4	2.1	4
Intel 2.4 GHz Xeon	0.3	4.3
Intel 1.4 GHz Itanium 2	1.8	6.4

Other Common Reasons:

- Threaded applications offer potential performance gains and practical advantages over non-threaded applications in several other ways:
 - Overlapping CPU work with I/O: For example, a program may have sections where it is performing a long I/O operation. While one thread is waiting for an I/O system call to complete, CPU intensive work can be performed by other threads.
 - Priority/real-time scheduling: tasks which are more important can be scheduled to supersede or interrupt lower priority tasks.
 - Asynchronous event handling: tasks which service events of indeterminate frequency and duration can be interleaved. For example, a web server can both transfer data from previous requests and manage the arrival of new requests.
- A perfect example is the typical web browser, where many interleaved tasks can be happening at the same time, and where tasks can vary in priority.
- Another good example is a modern operating system, which makes extensive use of threads. A screenshot of the MS Windows OS and applications using threads is shown below.

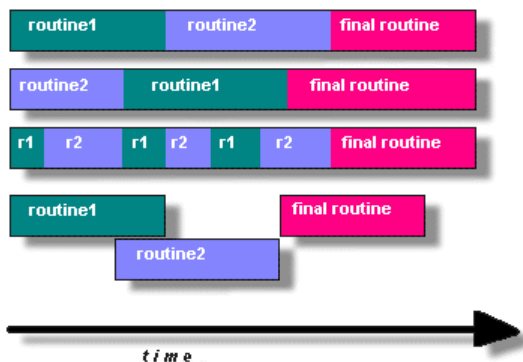


[Click for larger image](#)

Designing Threaded Programs

Parallel Programming:

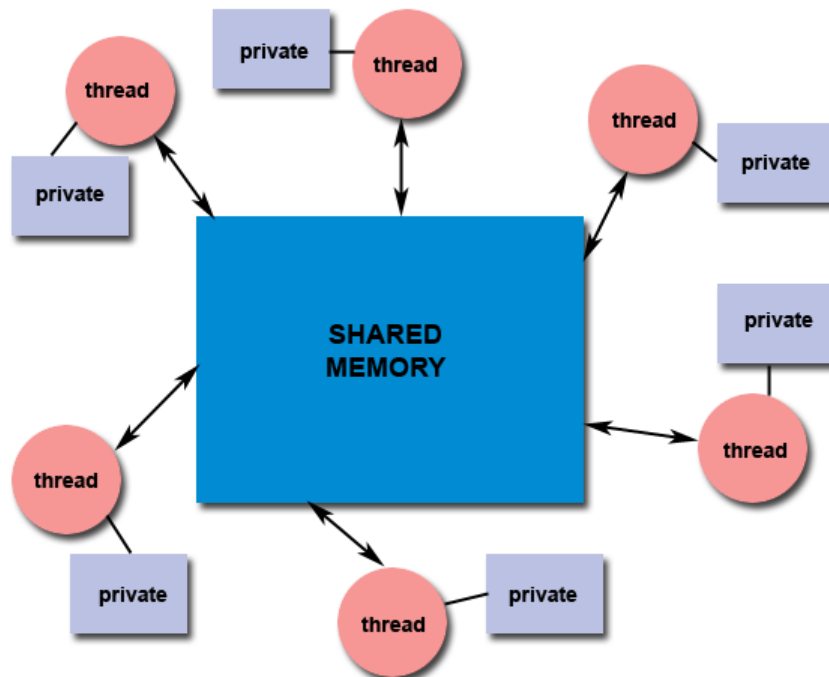
- On modern, multi-core machines, pthreads are ideally suited for parallel programming, and whatever applies to parallel programming in general, applies to parallel pthreads programs.
- There are many considerations for designing parallel programs, such as:
 - What type of parallel programming model to use?
 - Problem partitioning
 - Load balancing
 - Communications
 - Data dependencies
 - Synchronization and race conditions
 - Memory issues
 - I/O issues
 - Program complexity
 - Programmer effort/costs/time
 - ...
- Covering these topics is beyond the scope of this tutorial, however interested readers can obtain a quick overview in the [Introduction to Parallel Computing](#) tutorial.
- In general though, in order for a program to take advantage of Pthreads, it must be able to be organized into discrete, independent tasks which can execute concurrently. For example, if routine1 and routine2 can be interchanged, interleaved and/or overlapped in real time, they are candidates for threading.



- Programs having the following characteristics may be well suited for pthreads:
 - Work that can be executed, or data that can be operated on, by multiple tasks simultaneously:
 - Block for potentially long I/O waits
 - Use many CPU cycles in some places but not others
 - Must respond to asynchronous events
 - Some work is more important than other work (priority interrupts)
- Several common models for threaded programs exist:
 - Manager/worker:** a single thread, the *manager* assigns work to other threads, the *workers*. Typically, the manager handles all input and parcels out work to the other tasks. At least two forms of the manager/worker model are common: static worker pool and dynamic worker pool.
 - Pipeline:** a task is broken into a series of suboperations, each of which is handled in series, but concurrently, by a different thread. An automobile assembly line best describes this model.
 - Peer:** similar to the manager/worker model, but after the main thread creates other threads, it participates in the work.

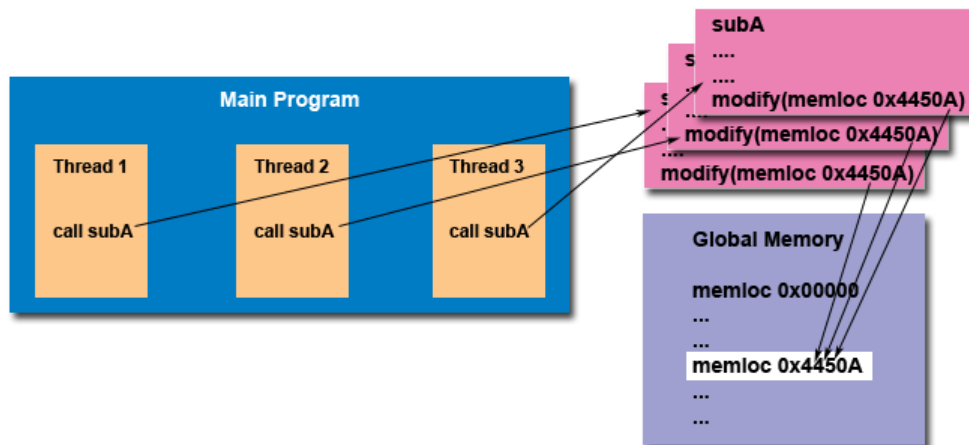
Shared Memory Model:

- All threads have access to the same global, shared memory
- Threads also have their own private data
- Programmers are responsible for synchronizing access (protecting) globally shared data.



Thread-safeness:

- Thread-safeness: in a nutshell, refers an application's ability to execute multiple threads simultaneously without "clobbering" shared data or creating "race" conditions.
- For example, suppose that your application creates several threads, each of which makes a call to the same library routine:
 - This library routine accesses/modifies a global structure or location in memory.
 - As each thread calls this routine it is possible that they may try to modify this global structure/memory location at the same time.
 - If the routine does not employ some sort of synchronization constructs to prevent data corruption, then it is not thread-safe.



- The implication to users of external library routines is that if you aren't 100% certain the routine is thread-safe, then you take your chances with problems that could arise.
- Recommendation: Be careful if your application uses libraries or other objects that don't explicitly guarantee thread-safeness. When in doubt, assume that they are not thread-safe until proven otherwise. This can be done by "serializing" the calls to the uncertain routine, etc.

Thread Limits:

- Although the Pthreads API is an ANSI/IEEE standard, implementations can, and usually do, vary in ways not specified by the standard.
- Because of this, a program that runs fine on one platform, may fail or produce wrong results on another platform.
- For example, the maximum number of threads permitted, and the default thread stack size are two important limits to consider when designing your program.
- Several thread limits are discussed in more detail later in this tutorial.

The Pthreads API

- The original Pthreads API was defined in the ANSI/IEEE POSIX 1003.1 - 1995 standard. The POSIX standard has continued to evolve and undergo revisions, including the Pthreads specification.
- Copies of the standard can be purchased from IEEE or downloaded for free from other sites online.
- The subroutines which comprise the Pthreads API can be informally grouped into four major groups:
 1. **Thread management:** Routines that work directly on threads - creating, detaching, joining, etc. They also include functions to set/query thread attributes

(joinable, scheduling etc.)

2. **Mutexes:** Routines that deal with synchronization, called a "mutex", which is an abbreviation for "mutual exclusion". Mutex functions provide for creating, destroying, locking and unlocking mutexes. These are supplemented by mutex attribute functions that set or modify attributes associated with mutexes.
3. **Condition variables:** Routines that address communications between threads that share a mutex. Based upon programmer specified conditions. This group includes functions to create, destroy, wait and signal based upon specified variable values. Functions to set/query condition variable attributes are also included.
4. **Synchronization:** Routines that manage read/write locks and barriers.

- Naming conventions: All identifiers in the threads library begin with **pthread_**. Some examples are shown below.

Routine Prefix	Functional Group
pthread_	Threads themselves and miscellaneous subroutines
pthread_attr_	Thread attributes objects
pthread_mutex_	Mutexes
pthread_mutexattr_	Mutex attributes objects.
pthread_cond_	Condition variables
pthread_condattr_	Condition attributes objects
pthread_key_	Thread-specific data keys
pthread_rwlock_	Read/write locks
pthread_barrier_	Synchronization barriers

- The concept of opaque objects pervades the design of the API. The basic calls work to create or modify opaque objects - the opaque objects can be modified by calls to attribute functions, which deal with opaque attributes.
- The Pthreads API contains around 100 subroutines. This tutorial will focus on a subset of these - specifically, those which are most likely to be immediately useful to the beginning Pthreads programmer.
- For portability, the **pthread.h** header file should be included in each source file using the Pthreads library.
- The current POSIX standard is defined only for the C language. Fortran programmers can use wrappers around C function calls. Some Fortran compilers may provide a Fortran pthreads API.
- A number of excellent books about Pthreads are available. Several of these are listed in the [References](#) section of this tutorial.

Compiling Threaded Programs

- Several examples of compile commands used for pthreads codes are listed in the table below.

Compiler / Platform	Compiler Command	Description
INTEL Linux	icc -pthread	C
	icpc -pthread	C++
PGI Linux	pgcc -lpthread	C
	pgCC -lpthread	C++
GNU Linux, Blue Gene	gcc -pthread	GNU C
	g++ -pthread	GNU C++
IBM Blue Gene	bgxlc_r / bgcc_r	C (ANSI / non-ANSI)
	bgxlc_r, bgxlc++_r	C++

Thread Management

Creating and Terminating Threads

► Routines:

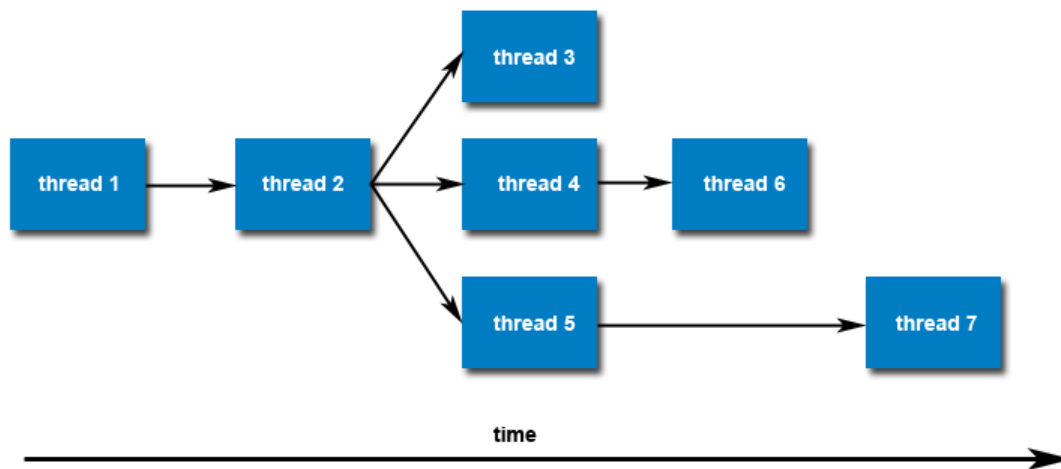
```
pthread_create (thread,attr,start_routine,arg)
pthread_exit (status)
pthread_cancel (thread)
pthread_attr_init (attr)
pthread_attr_destroy (attr)
```

► Creating Threads:

- Initially, your `main()` program comprises a single, default thread. All other threads must be explicitly created by the programmer.
- `pthread_create` creates a new thread and makes it executable. This routine can be called any number of times from anywhere within your code.
- `pthread_create` arguments:
 - `thread`: An opaque, unique identifier for the new thread returned by the subroutine.
 - `attr`: An opaque attribute object that may be used to set thread attributes. You can specify a thread attributes object, or NULL for the default values.
 - `start_routine`: the C routine that the thread will execute once it is created.
 - `arg`: A single argument that may be passed to `start_routine`. It must be passed by reference as a pointer cast of type void. NULL may be used if no argument is to be passed.
- The maximum number of threads that may be created by a process is implementation dependent. Programs that attempt to exceed the limit can fail or produce wrong results.
- Querying and setting your implementation's thread limit - Linux example shown. Demonstrates querying the default (soft) limits and then setting the maximum number of processes (including threads) to the hard limit. Then verifying that the limit has been overridden.

bash / ksh / sh	tcsh / csh
<pre>\$ ulimit -a core file size (blocks, -c) 16 data seg size (kbytes, -d) unlimited scheduling priority (-e) 0 file size (blocks, -f) unlimited pending signals (-i) 255956 max locked memory (kbytes, -l) 64 max memory size (kbytes, -m) unlimited open files (-n) 1024 pipe size (512 bytes, -p) 8 POSIX message queues (bytes, -q) 819200 real-time priority (-r) 0 stack size (kbytes, -s) unlimited cpu time (seconds, -t) unlimited max user processes (-u) 1024 virtual memory (kbytes, -v) unlimited file locks (-x) unlimited \$ ulimit -Hu 7168 \$ ulimit -u 7168 \$ ulimit -a core file size (blocks, -c) 16 data seg size (kbytes, -d) unlimited scheduling priority (-e) 0 file size (blocks, -f) unlimited pending signals (-i) 255956 max locked memory (kbytes, -l) 64 max memory size (kbytes, -m) unlimited open files (-n) 1024 pipe size (512 bytes, -p) 8 POSIX message queues (bytes, -q) 819200 real-time priority (-r) 0 stack size (kbytes, -s) unlimited cpu time (seconds, -t) unlimited max user processes (-u) 7168 virtual memory (kbytes, -v) unlimited file locks (-x) unlimited</pre>	<pre>% limit cputime unlimited filesize unlimited datasize unlimited stacksize unlimited coredumpsize 16 kbytes memoryuse unlimited vmemoryuse unlimited descriptors 1024 memorylocked 64 kbytes maxproc 1024 % limit maxproc unlimited % limit cputime unlimited filesize unlimited datasize unlimited stacksize unlimited coredumpsize 16 kbytes memoryuse unlimited vmemoryuse unlimited descriptors 1024 memorylocked 64 kbytes maxproc 7168</pre>

- Once created, threads are peers, and may create other threads. There is no implied hierarchy or dependency between threads.



Thread Attributes:

- By default, a thread is created with certain attributes. Some of these attributes can be changed by the programmer via the thread attribute object.
- `pthread_attr_init` and `pthread_attr_destroy` are used to initialize/destroy the thread attribute object.
- Other routines are then used to query/set specific attributes in the thread attribute object. Attributes include:
 - Detached or joinable state
 - Scheduling inheritance
 - Scheduling policy
 - Scheduling parameters
 - Scheduling contention scope
 - Stack size
 - Stack address
 - Stack guard (overflow) size
- Some of these attributes will be discussed later.

Thread Binding and Scheduling:



Question: After a thread has been created, how do you know a)when it will be scheduled to run by the operating system, and b)which processor/core it will run on?

Answer

- The Pthreads API provides several routines that may be used to specify how threads are scheduled for execution. For example, threads can be scheduled to run FIFO (first-in first-out), RR (round-robin) or OTHER (operating system determines). It also provides the ability to set a thread's scheduling priority value.
- These topics are not covered here, however a good overview of "how things work" under Linux can be found in the [sched_setscheduler](#) man page.
- The Pthreads API does not provide routines for binding threads to specific cpus/cores. However, local implementations may include this functionality - such as providing the non-standard `pthread_setaffinity_np` routine. Note that "_np" in the name stands for "non-portable".
- Also, the local operating system may provide a way to do this. For example, Linux provides the `sched_setaffinity` routine.

Terminating Threads & `pthread_exit()`:

- There are several ways in which a thread may be terminated:
 - The thread returns normally from its starting routine. Its work is done.
 - The thread makes a call to the `pthread_exit` subroutine - whether its work is done or not.
 - The thread is canceled by another thread via the `pthread_cancel` routine.
 - The entire process is terminated due to making a call to either the `exec()` or `exit()`
 - If `main()` finishes first, without calling `pthread_exit` explicitly itself
- The `pthread_exit()` routine allows the programmer to specify an optional termination *status* parameter. This optional parameter is typically returned to threads "joining" the terminated thread (covered later).
- In subroutines that execute to completion normally, you can often dispense with calling `pthread_exit()` - unless, of course, you want to pass the optional status code back.
- Cleanup: the `pthread_exit()` routine does not close files; any files opened inside the thread will remain open after the thread is terminated.
- Discussion on calling `pthread_exit()` from `main()`:
 - There is a definite problem if `main()` finishes before the threads it spawned if you don't call `pthread_exit()` explicitly. All of the threads it created will terminate because `main()` is done and no longer exists to support the threads.
 - By having `main()` explicitly call `pthread_exit()` as the last thing it does, `main()` will block and be kept alive to support the threads it created until they are done.

Example: Pthread Creation and Termination

- This simple example code creates 5 threads with the `pthread_create()` routine. Each thread prints a "Hello World!" message, and then terminates with a call to `pthread_exit()`.



Thread Creation and Termination Example

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #define NUM_THREADS      5
4
5  void *PrintHello(void *threadid)
6  {
7      long tid;
8      tid = (long)threadid;
9      printf("Hello World! It's me, thread #%ld!\n", tid);
10     pthread_exit(NULL);
11 }
12
13 int main (int argc, char *argv[])
14 {
15     pthread_t threads[NUM_THREADS];
16     int rc;
17     long t;
18     for(t=0; t<NUM_THREADS; t++){
19         printf("In main: creating thread %ld\n", t);
20         rc = pthread_create(&threads[t], NULL, PrintHello, (void *)t);
21         if (rc){
22             printf("ERROR; return code from pthread_create() is %d\n", rc);
23             exit(-1);
24         }
25     }
26
27     /* Last thing that main() should do */
28     pthread_exit(NULL);
29 }

```

[Source](#)[Output](#)

Thread Management

Passing Arguments to Threads

- The `pthread_create()` routine permits the programmer to pass one argument to the thread start routine. For cases where multiple arguments must be passed, this limitation is easily overcome by creating a structure which contains all of the arguments, and then passing a pointer to that structure in the `pthread_create()` routine.
- All arguments must be passed by reference and cast to (void *).



Question: How can you safely pass data to newly created threads, given their non-deterministic start-up and scheduling?

[Answer](#)

Example 1 - Thread Argument Passing

This code fragment demonstrates how to pass a simple integer to each thread. The calling thread uses a unique data structure for each thread, insuring that each thread's argument remains intact throughout the program.

```

long taskids[NUM_THREADS];

for(t=0; t<NUM_THREADS; t++)
{
    taskids[t] = t;
    printf("Creating thread %ld\n", t);
    rc = pthread_create(&threads[t], NULL, PrintHello, (void *) taskids[t]);
    ...
}

```

[Source](#)[Output](#)

Example 2 - Thread Argument Passing

This example shows how to setup/pass multiple arguments via a structure. Each thread receives a unique instance of the structure.

```

struct thread_data{
    int  thread_id;
    int  sum;
    char *message;
};

struct thread_data thread_data_array[NUM_THREADS];

void *PrintHello(void *threadarg)
{
    struct thread_data *my_data;
    ...
    my_data = (struct thread_data *) threadarg;
    taskid = my_data->thread_id;
    sum = my_data->sum;
    hello_msg = my_data->message;
    ...
}

int main (int argc, char *argv[])
{
    ...
    thread_data_array[t].thread_id = t;
    thread_data_array[t].sum = sum;
    thread_data_array[t].message = messages[t];
    rc = pthread_create(&threads[t], NULL, PrintHello,
        (void *) &thread_data_array[t]);
    ...
}

```

[Source](#) [Output](#)



Example 3 - Thread Argument Passing (Incorrect)

This example performs argument passing incorrectly. It passes the *address* of variable `t`, which is shared memory space and visible to all threads. As the loop iterates, the value of this memory location changes, possibly before the created threads can access it.

```

int rc;
long t;

for(t=0; t<NUM_THREADS; t++)
{
    printf("Creating thread %ld\n", t);
    rc = pthread_create(&threads[t], NULL, PrintHello, (void *) &t);
    ...
}

```

[Source](#) [Output](#)

Thread Management

Joining and Detaching Threads

▶ Routines:

```

pthread_join (threadid,status)

pthread_detach (threadid)

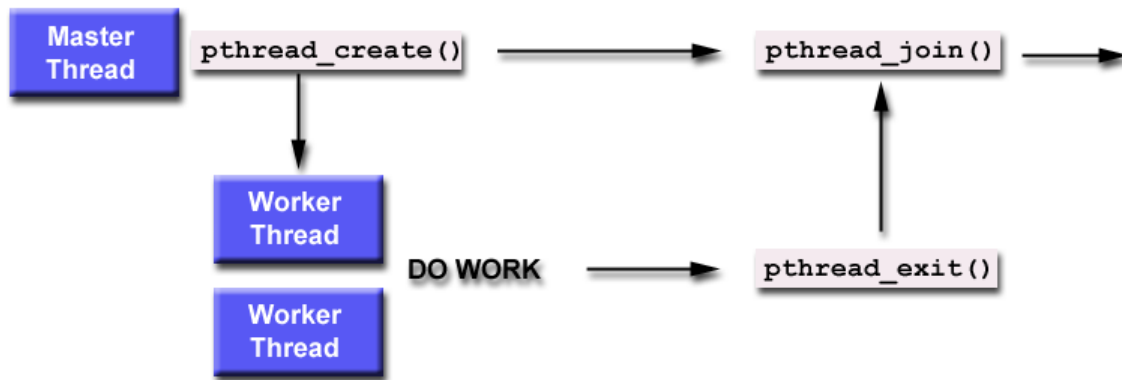
pthread_attr_setdetachstate (attr,detachstate)

pthread_attr_getdetachstate (attr,detachstate)

```

▶ Joining:

- "Joining" is one way to accomplish synchronization between threads. For example:



- The `pthread_join()` subroutine blocks the calling thread until the specified `threadid` thread terminates.
- The programmer is able to obtain the target thread's termination return `status` if it was specified in the target thread's call to `pthread_exit()`.
- A joining thread can match one `pthread_join()` call. It is a logical error to attempt multiple joins on the same thread.
- Two other synchronization methods, mutexes and condition variables, will be discussed later.

Joinable or Not?

- When a thread is created, one of its attributes defines whether it is joinable or detached. Only threads that are created as joinable can be joined. If a thread is created as detached, it can never be joined.
- The final draft of the POSIX standard specifies that threads should be created as joinable.
- To explicitly create a thread as joinable or detached, the `attr` argument in the `pthread_create()` routine is used. The typical 4 step process is:
 1. Declare a pthread attribute variable of the `pthread_attr_t` data type
 2. Initialize the attribute variable with `pthread_attr_init()`
 3. Set the attribute detached status with `pthread_attr_setdetachstate()`
 4. When done, free library resources used by the attribute with `pthread_attr_destroy()`

Detaching:

- The `pthread_detach()` routine can be used to explicitly detach a thread even though it was created as joinable.
- There is no converse routine.

Recommendations:

- If a thread requires joining, consider explicitly creating it as joinable. This provides portability as not all implementations may create threads as joinable by default.
- If you know in advance that a thread will never need to join with another thread, consider creating it in a detached state. Some system resources may be able to be freed.

Example: Pthread Joining

- This example demonstrates how to "wait" for thread completions by using the Pthread join routine.
- Since some implementations of Pthreads may not create threads in a joinable state, the threads in this example are explicitly created in a joinable state so that they can be joined later.

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #define NUM_THREADS    4
6
7  void *BusyWork(void *t)
8  {
9      int i;
10     long tid;
11     double result=0.0;
12     tid = (long)t;
13     printf("Thread %ld starting...\n",tid);
14     for (i=0; i<1000000; i++)
15     {
16         result = result + sin(i) * tan(i);
17     }
18     printf("Thread %ld done. Result = %e\n",tid, result);
19     pthread_exit((void*) t);
20 }
  
```

```

21
22 int main (int argc, char *argv[])
23 {
24     pthread_t thread[NUM_THREADS];
25     pthread_attr_t attr;
26     int rc;
27     long t;
28     void *status;
29
30     /* Initialize and set thread detached attribute */
31     pthread_attr_init(&attr);
32     pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
33
34     for(t=0; t<NUM_THREADS; t++) {
35         printf("Main: creating thread %ld\n", t);
36         rc = pthread_create(&thread[t], &attr, BusyWork, (void *)t);
37         if (rc) {
38             printf("ERROR; return code from pthread_create() is %d\n", rc);
39             exit(-1);
40         }
41     }
42
43     /* Free attribute and wait for the other threads */
44     pthread_attr_destroy(&attr);
45     for(t=0; t<NUM_THREADS; t++) {
46         rc = pthread_join(thread[t], &status);
47         if (rc) {
48             printf("ERROR; return code from pthread_join() is %d\n", rc);
49             exit(-1);
50         }
51         printf("Main: completed join with thread %ld having a status\n", t, (long)status);
52     }
53
54     printf("Main: program completed. Exiting.\n");
55     pthread_exit(NULL);
56 }
57

```

[Source](#) [Output](#)

Thread Management

Stack Management

▶ Routines:

```

pthread_attr_getstacksize (attr, stacksize)

pthread_attr_setstacksize (attr, stacksize)

pthread_attr_getstackaddr (attr, stackaddr)

pthread_attr_setstackaddr (attr, stackaddr)

```

▶ Preventing Stack Problems:

- The POSIX standard does not dictate the size of a thread's stack. This is implementation dependent and varies.
- Exceeding the default stack limit is often very easy to do, with the usual results: program termination and/or corrupted data.
- Safe and portable programs do not depend upon the default stack limit, but instead, explicitly allocate enough stack for each thread by using the `pthread_attr_setstacksize` routine.
- The `pthread_attr_getstackaddr` and `pthread_attr_setstackaddr` routines can be used by applications in an environment where the stack for a thread must be placed in some particular region of memory.

▶ Some Practical Examples at LC:

- Default thread stack size varies greatly. The maximum size that can be obtained also varies greatly, and may depend upon the number of threads per node.
- Both past and present architectures are shown to demonstrate the wide variation in default thread stack size.

Node Architecture	#CPUs	Memory (GB)	Default Size (bytes)
Intel Xeon E5-2670	16	32	2,097,152
Intel Xeon 5660	12	24	2,097,152
AMD Opteron	8	16	2,097,152
Intel IA64	4	8	33,554,432
Intel IA32	2	4	2,097,152
IBM Power5	8	32	196,608
IBM Power4	8	16	196,608
IBM Power3	16	16	98,304

Example: Stack Management

- This example demonstrates how to query and set a thread's stack size.



Stack Management Example

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #define NTHREADS 4
4  #define N 1000
5  #define MEGEXTRA 1000000
6
7  pthread_attr_t attr;
8
9  void *dowork(void *threadid)
10 {
11     double A[N][N];
12     int i,j;
13     long tid;
14     size_t mystacksize;
15
16     tid = (long)threadid;
17     pthread_attr_getstacksize (&attr, &mystacksize);
18     printf("Thread %ld: stack size = %li bytes \n", tid, mystacksize);
19     for (i=0; i<N; i++)
20         for (j=0; j<N; j++)
21             A[i][j] = ((i*j)/3.452) + (N-i);
22     pthread_exit(NULL);
23 }
24
25 int main(int argc, char *argv[])
26 {
27     pthread_t threads[NTHREADS];
28     size_t stacksize;
29     int rc;
30     long t;
31
32     pthread_attr_init(&attr);
33     pthread_attr_getstacksize (&attr, &stacksize);
34     printf("Default stack size = %li\n", stacksize);
35     stacksize = sizeof(double)*N*N+MEGEXTRA;
36     printf("Amount of stack needed per thread = %li\n",stacksize);
37     pthread_attr_setstacksize (&attr, stacksize);
38     printf("Creating threads with stack size = %li bytes\n",stacksize);
39     for(t=0; t<NTHREADS; t++){
40         rc = pthread_create(&threads[t], &attr, dowork, (void *)t);
41         if (rc){
42             printf("ERROR; return code from pthread_create() is %d\n", rc);
43             exit(-1);
44         }
45     }
46     printf("Created %ld threads.\n", t);
47     pthread_exit(NULL);
48 }

```

Thread Management

Miscellaneous Routines

`pthread_self ()`
`pthread_equal (thread1,thread2)`

- `pthread_self` returns the unique, system assigned thread ID of the calling thread.
- `pthread_equal` compares two thread IDs. If the two IDs are different 0 is returned, otherwise a non-zero value is returned.
- Note that for both of these routines, the thread identifier objects are opaque and can not be easily inspected. Because thread IDs are opaque objects, the C language equivalence operator `==` should not be used to compare two thread IDs against each other, or to compare a single thread ID against another value.

`pthread_once (once_control, init_routine)`

- `pthread_once` executes the `init_routine` exactly once in a process. The first call to this routine by any thread in the process executes the given `init_routine`, without parameters. Any subsequent call will have no effect.
- The `init_routine` routine is typically an initialization routine.
- The `once_control` parameter is a synchronization control structure that requires initialization prior to calling `pthread_once`. For example:

`pthread_once_t once_control = PTHREAD_ONCE_INIT;`

Pthread Exercise 1

Getting Started and Thread Management Routines

Overview:

- Login to an LC cluster using your workshop username and OTP token
- Copy the exercise files to your home directory
- Familiarize yourself with LC's Pthreads environment
- Write a simple "Hello World" Pthreads program
- Successfully compile your program
- Successfully run your program - several different ways
- Review, compile, run and/or debug some related Pthreads programs (provided)



[GO TO THE EXERCISE HERE](#)

Mutex Variables

Overview

- Mutex is an abbreviation for "mutual exclusion". Mutex variables are one of the primary means of implementing thread synchronization and for protecting shared data when multiple writes occur.
- A mutex variable acts like a "lock" protecting access to a shared data resource. The basic concept of a mutex as used in Pthreads is that only one thread can lock (or own) a mutex variable at any given time. Thus, even if several threads try to lock a mutex only one thread will be successful. No other thread can own that mutex until the owning thread unlocks that mutex. Threads must "take turns" accessing protected data.
- Mutexes can be used to prevent "race" conditions. An example of a race condition involving a bank transaction is shown below:

Thread 1	Thread 2	Balance
Read balance: \$1000		\$1000

	Read balance: \$1000	\$1000
	Deposit \$200	\$1000
Deposit \$200		\$1000
Update balance \$1000+\$200		\$1200
	Update balance \$1000+\$200	\$1200

- In the above example, a mutex should be used to lock the "Balance" while a thread is using this shared data resource.
- Very often the action performed by a thread owning a mutex is the updating of global variables. This is a safe way to ensure that when several threads update the same variable, the final value is the same as what it would be if only one thread performed the update. The variables being updated belong to a "critical section".
- A typical sequence in the use of a mutex is as follows:
 - Create and initialize a mutex variable
 - Several threads attempt to lock the mutex
 - Only one succeeds and that thread owns the mutex
 - The owner thread performs some set of actions
 - The owner unlocks the mutex
 - Another thread acquires the mutex and repeats the process
 - Finally the mutex is destroyed
- When several threads compete for a mutex, the losers block at that call - an unblocking call is available with "trylock" instead of the "lock" call.
- When protecting shared data, it is the programmer's responsibility to make sure every thread that needs to use a mutex does so. For example, if 4 threads are updating the same data, but only one uses a mutex, the data can still be corrupted.

Mutex Variables

Creating and Destroying Mutexes

▶ Routines:

```
pthread_mutex_init (mutex,attr)

pthread_mutex_destroy (mutex)

pthread_mutexattr_init (attr)

pthread_mutexattr_destroy (attr)
```

▶ Usage:

- Mutex variables must be declared with type `pthread_mutex_t`, and must be initialized before they can be used. There are two ways to initialize a mutex variable:
 1. Statically, when it is declared. For example:


```
pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;
```
 2. Dynamically, with the `pthread_mutex_init()` routine. This method permits setting mutex object attributes, *attr*.

The mutex is initially unlocked.
- The *attr* object is used to establish properties for the mutex object, and must be of type `pthread_mutexattr_t` if used (may be specified as NULL to accept defaults). The Pthreads standard defines three optional mutex attributes:
 - Protocol: Specifies the protocol used to prevent priority inversions for a mutex.
 - Prioceiling: Specifies the priority ceiling of a mutex.
 - Process-shared: Specifies the process sharing of a mutex.

Note that not all implementations may provide the three optional mutex attributes.
- The `pthread_mutexattr_init()` and `pthread_mutexattr_destroy()` routines are used to create and destroy mutex attribute objects respectively.
- `pthread_mutex_destroy()` should be used to free a mutex object which is no longer needed.

Mutex Variables

Locking and Unlocking Mutexes

▶ Routines:

```
pthread_mutex_lock (mutex)

pthread_mutex_trylock (mutex)

pthread_mutex_unlock (mutex)
```

▶ Usage:

- The `pthread_mutex_lock()` routine is used by a thread to acquire a lock on the specified *mutex* variable. If the mutex is already locked by another thread, this call will block the calling thread until the mutex is unlocked.

- `pthread_mutex_trylock()` will attempt to lock a mutex. However, if the mutex is already locked, the routine will return immediately with a "busy" error code. This routine may be useful in preventing deadlock conditions, as in a priority-inversion situation.
- `pthread_mutex_unlock()` will unlock a mutex if called by the owning thread. Calling this routine is required after a thread has completed its use of protected data if other threads are to acquire the mutex for their work with the protected data. An error will be returned if:
 - If the mutex was already unlocked
 - If the mutex is owned by another thread
- There is nothing "magical" about mutexes...in fact they are akin to a "gentlemen's agreement" between participating threads. It is up to the code writer to insure that the necessary threads all make the the mutex lock and unlock calls correctly. The following scenario demonstrates a logical error:

Thread 1	Thread 2	Thread 3
Lock	Lock	
A = 2	A = A+1	A = A*B
Unlock	Unlock	



Question: When more than one thread is waiting for a locked mutex, which thread will be granted the lock first after it is released?

Answer

Example: Using Mutexes

- This example program illustrates the use of mutex variables in a threads program that performs a dot product.
- The main data is made available to all threads through a globally accessible structure.
- Each thread works on a different part of the data.
- The main thread waits for all the threads to complete their computations, and then it prints the resulting sum.



Using Mutexes Example

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  /*
6   The following structure contains the necessary information
7   to allow the function "dotprod" to access its input data and
8   place its output into the structure.
9   */
10
11 typedef struct
12 {
13     double    *a;
14     double    *b;
15     double    sum;
16     int       vecLen;
17 } DOTDATA;
18
19 /* Define globally accessible variables and a mutex */
20
21 #define NUMTHRDS 4
22 #define VECLLEN 100
23 DOTDATA dotstr;
24 pthread_t callThd[NUMTHRDS];
25 pthread_mutex_t mutexsum;
26
27 /*
28 The function dotprod is activated when the thread is created.
29 All input to this routine is obtained from a structure
30 of type DOTDATA and all output from this function is written into
31 this structure. The benefit of this approach is apparent for the
32 multi-threaded program: when a thread is created we pass a single
33 argument to the activated function - typically this argument
34 is a thread number. All the other information required by the
35 function is accessed from the globally accessible structure.
36 */
37
38 void *dotprod(void *arg)
39 {

```



```

40
41     /* Define and use local variables for convenience */
42
43     int i, start, end, len ;
44     long offset;
45     double mysum, *x, *y;
46     offset = (long)arg;
47
48     len = dotstr.vecLEN;
49     start = offset*len;
50     end   = start + len;
51     x = dotstr.a;
52     y = dotstr.b;
53
54     /*
55     Perform the dot product and assign result
56     to the appropriate variable in the structure.
57     */
58
59     mysum = 0;
60     for (i=start; i<end ; i++)
61     {
62         mysum += (x[i] * y[i]);
63     }
64
65     /*
66     Lock a mutex prior to updating the value in the shared
67     structure, and unlock it upon updating.
68     */
69     pthread_mutex_lock (&mutexsum);
70     dotstr.sum += mysum;
71     pthread_mutex_unlock (&mutexsum);
72
73     pthread_exit((void*) 0);
74 }
75
76 /*
77 The main program creates threads which do all the work and then
78 print out result upon completion. Before creating the threads,
79 the input data is created. Since all threads update a shared structure,
80 we need a mutex for mutual exclusion. The main thread needs to wait for
81 all threads to complete, it waits for each one of the threads. We specify
82 a thread attribute value that allow the main thread to join with the
83 threads it creates. Note also that we free up handles when they are
84 no longer needed.
85 */
86
87 int main (int argc, char *argv[])
88 {
89     long i;
90     double *a, *b;
91     void *status;
92     pthread_attr_t attr;
93
94     /* Assign storage and initialize values */
95     a = (double*) malloc (NUMTHRDS*VECLEN*sizeof(double));
96     b = (double*) malloc (NUMTHRDS*VECLEN*sizeof(double));
97
98     for (i=0; i<VECLEN*NUMTHRDS; i++)
99     {
100         a[i]=1.0;
101         b[i]=a[i];
102     }
103
104     dotstr.vecLEN = VECLN;
105     dotstr.a = a;

```

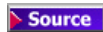
```

106     dotstr.b = b;
107     dotstr.sum=0;
108
109     pthread_mutex_init(&mutexsum, NULL);
110
111     /* Create threads to perform the dotproduct */
112     pthread_attr_init(&attr);
113     pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
114
115     for(i=0; i<NUMTHRDS; i++)
116     {
117         /*
118         Each thread works on a different set of data. The offset is specified
119         by 'i'. The size of the data for each thread is indicated by VECLEN.
120         */
121         pthread_create(&callThd[i], &attr, dotprod, (void *)i);
122     }
123
124     pthread_attr_destroy(&attr);
125
126     /* Wait on the other threads */
127     for(i=0; i<NUMTHRDS; i++)
128     {
129         pthread_join(callThd[i], &status);
130     }
131
132     /* After joining, print out the results and cleanup */
133     printf ("Sum = %f \n", dotstr.sum);
134     free (a);
135     free (b);
136     pthread_mutex_destroy(&mutexsum);
137     pthread_exit(NULL);
138 }

```



Serial version



Pthreads version

Condition Variables

Overview

- Condition variables provide yet another way for threads to synchronize. While mutexes implement synchronization by controlling thread access to data, condition variables allow threads to synchronize based upon the actual value of data.
- Without condition variables, the programmer would need to have threads continually polling (possibly in a critical section), to check if the condition is met. This can be very resource consuming since the thread would be continuously busy in this activity. A condition variable is a way to achieve the same goal without polling.
- A condition variable is always used in conjunction with a mutex lock.
- A representative sequence for using condition variables is shown below.

Main Thread

- Declare and initialize global data/variables which require synchronization (such as "count")
- Declare and initialize a condition variable object
- Declare and initialize an associated mutex
- Create threads A and B to do work

Thread A

- Do work up to the point where a certain condition must occur (such as "count" must reach a specified value)
- Lock associated mutex and check value of a global variable
- Call `pthread_cond_wait()` to perform a blocking wait for signal from Thread-B. Note that a call to `pthread_cond_wait()` automatically and atomically unlocks the associated mutex variable so that it can be used by Thread-B.
- When signalled, wake up. Mutex is automatically and atomically locked.
- Explicitly unlock mutex
- Continue

Thread B

- Do work
- Lock associated mutex
- Change the value of the global variable that Thread-A is waiting upon.
- Check value of the global Thread-A wait variable. If it fulfills the desired condition, signal Thread-A.
- Unlock mutex.
- Continue

Main Thread

Join / Continue

Condition Variables

Creating and Destroying Condition Variables

▶ Routines:

```
pthread_cond_init (condition,attr)

pthread_cond_destroy (condition)

pthread_condattr_init (attr)

pthread_condattr_destroy (attr)
```

▶ Usage:

- Condition variables must be declared with type `pthread_cond_t`, and must be initialized before they can be used. There are two ways to initialize a condition variable:
 1. Statically, when it is declared. For example:

```
pthread_cond_t myconvar = PTHREAD_COND_INITIALIZER;
```
 2. Dynamically, with the `pthread_cond_init()` routine. The ID of the created condition variable is returned to the calling thread through the *condition* parameter. This method permits setting condition variable object attributes, *attr*.
- The optional *attr* object is used to set condition variable attributes. There is only one attribute defined for condition variables: process-shared, which allows the condition variable to be seen by threads in other processes. The attribute object, if used, must be of type `pthread_condattr_t` (may be specified as NULL to accept defaults).

Note that not all implementations may provide the process-shared attribute.
- The `pthread_condattr_init()` and `pthread_condattr_destroy()` routines are used to create and destroy condition variable attribute objects.
- `pthread_cond_destroy()` should be used to free a condition variable that is no longer needed.

Condition Variables

Waiting and Signaling on Condition Variables

▶ Routines:

```
pthread_cond_wait (condition,mutex)

pthread_cond_signal (condition)

pthread_cond_broadcast (condition)
```

▶ Usage:

- `pthread_cond_wait()` blocks the calling thread until the specified *condition* is signalled. This routine should be called while *mutex* is locked, and it will automatically release the mutex while it waits. After signal is received and thread is awakened, *mutex* will be automatically locked for use by the thread. The programmer is then responsible for unlocking *mutex* when the thread is finished with it.

Recommendation: Using a WHILE loop instead of an IF statement (see `watch_count` routine in example below) to check the waited for condition can help deal with several potential problems, such as:

 - If several threads are waiting for the same wake up signal, they will take turns acquiring the mutex, and any one of them can then modify the condition they all waited for.
 - If the thread received the signal in error due to a program bug
 - The Pthreads library is permitted to issue spurious wake ups to a waiting thread without violating the standard.
- The `pthread_cond_signal()` routine is used to signal (or wake up) another thread which is waiting on the condition variable. It should be called after *mutex* is locked, and must unlock *mutex* in order for `pthread_cond_wait()` routine to complete.
- The `pthread_cond_broadcast()` routine should be used instead of `pthread_cond_signal()` if more than one thread is in a blocking wait state.
- It is a logical error to call `pthread_cond_signal()` before calling `pthread_cond_wait()`.



Proper locking and unlocking of the associated mutex variable is essential when using these routines. For example:

- Failing to lock the mutex before calling `pthread_cond_wait()` may cause it NOT to block.
- Failing to unlock the mutex after calling `pthread_cond_signal()` may not allow a matching `pthread_cond_wait()` routine to complete (it will remain blocked).

Example: Using Condition Variables

- This simple example code demonstrates the use of several Pthread condition variable routines.
- The main routine creates three threads.
- Two of the threads perform work and update a "count" variable.

- The third thread waits until the count variable reaches a specified value.



Using Condition Variables Example

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  #define NUM_THREADS 3
6  #define TCOUNT 10
7  #define COUNT_LIMIT 12
8
9  int    count = 0;
10 int    thread_ids[3] = {0,1,2};
11 pthread_mutex_t count_mutex;
12 pthread_cond_t count_threshold_cv;
13
14 void *inc_count(void *t)
15 {
16     int i;
17     long my_id = (long)t;
18
19     for (i=0; i<TCOUNT; i++) {
20         pthread_mutex_lock(&count_mutex);
21         count++;
22
23         /*
24          Check the value of count and signal waiting thread when condition is
25          reached. Note that this occurs while mutex is locked.
26          */
27         if (count == COUNT_LIMIT) {
28             pthread_cond_signal(&count_threshold_cv);
29             printf("inc_count(): thread %ld, count = %d Threshold reached.\n",
30                 my_id, count);
31         }
32         printf("inc_count(): thread %ld, count = %d, unlocking mutex\n",
33             my_id, count);
34         pthread_mutex_unlock(&count_mutex);
35
36         /* Do some "work" so threads can alternate on mutex lock */
37         sleep(1);
38     }
39     pthread_exit(NULL);
40 }
41
42 void *watch_count(void *t)
43 {
44     long my_id = (long)t;
45
46     printf("Starting watch_count(): thread %ld\n", my_id);
47
48     /*
49     Lock mutex and wait for signal. Note that the pthread_cond_wait
50     routine will automatically and atomically unlock mutex while it waits.
51     Also, note that if COUNT_LIMIT is reached before this routine is run by
52     the waiting thread, the loop will be skipped to prevent pthread_cond_wait
53     from never returning.
54     */
55     pthread_mutex_lock(&count_mutex);
56     while (count<COUNT_LIMIT) {
57         pthread_cond_wait(&count_threshold_cv, &count_mutex);
58         printf("watch_count(): thread %ld Condition signal received.\n", my_id);
59     }
60     count += 125;
61     printf("watch_count(): thread %ld count now = %d.\n", my_id, count);
62     pthread_mutex_unlock(&count_mutex);
63     pthread_exit(NULL);
```

```

64 }
65
66 int main (int argc, char *argv[])
67 {
68     int i, rc;
69     long t1=1, t2=2, t3=3;
70     pthread_t threads[3];
71     pthread_attr_t attr;
72
73     /* Initialize mutex and condition variable objects */
74     pthread_mutex_init(&count_mutex, NULL);
75     pthread_cond_init (&count_threshold_cv, NULL);
76
77     /* For portability, explicitly create threads in a joinable state */
78     pthread_attr_init(&attr);
79     pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
80     pthread_create(&threads[0], &attr, watch_count, (void *)t1);
81     pthread_create(&threads[1], &attr, inc_count, (void *)t2);
82     pthread_create(&threads[2], &attr, inc_count, (void *)t3);
83
84     /* Wait for all threads to complete */
85     for (i=0; i<NUM_THREADS; i++) {
86         pthread_join(threads[i], NULL);
87     }
88     printf ("Main(): Waited on %d  threads. Done.\n", NUM_THREADS);
89
90     /* Clean up and exit */
91     pthread_attr_destroy(&attr);
92     pthread_mutex_destroy(&count_mutex);
93     pthread_cond_destroy(&count_threshold_cv);
94     pthread_exit(NULL);
95
96 }

```

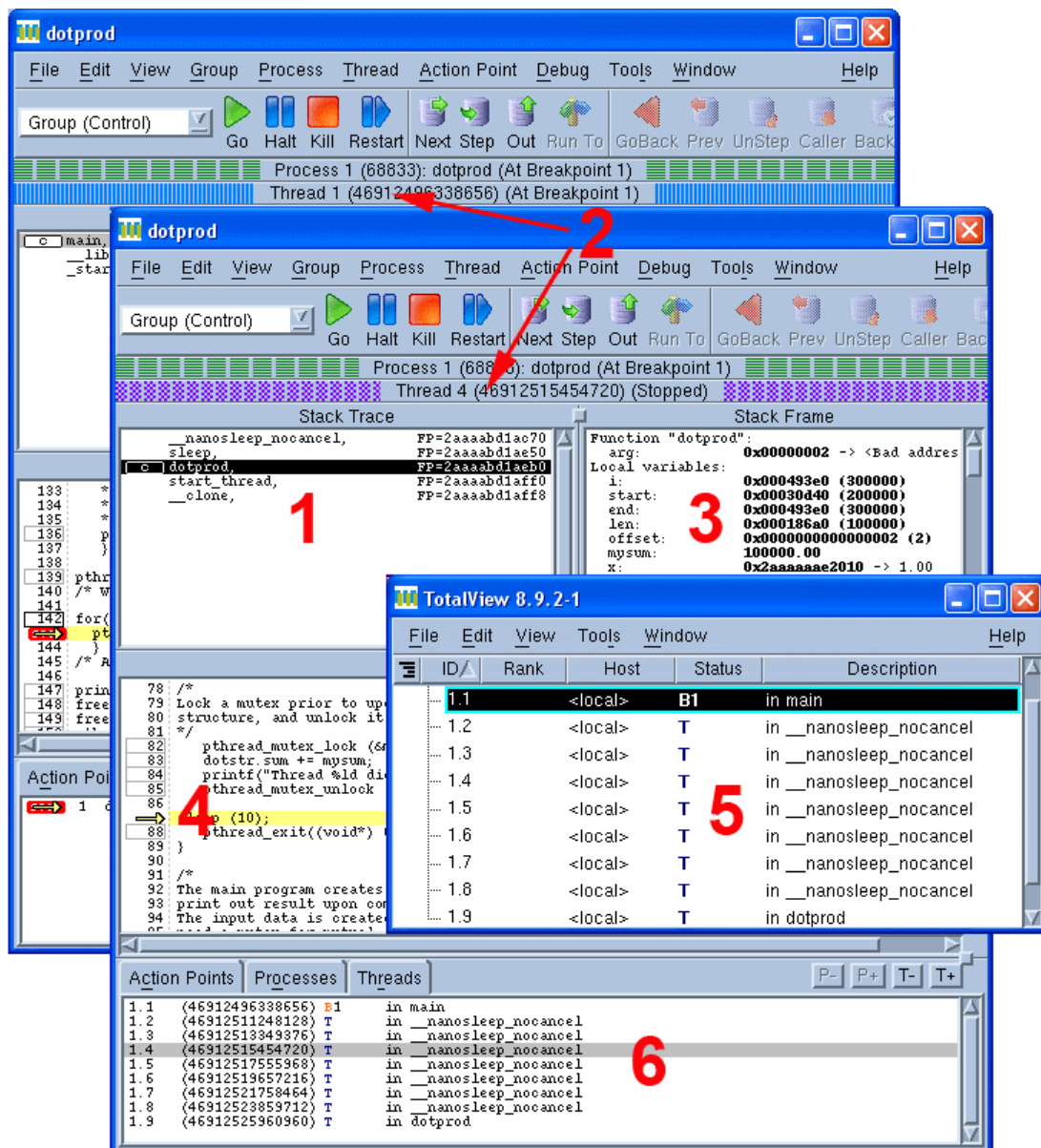
 Source

 Output

Monitoring, Debugging and Performance Analysis Tools for Pthreads

Monitoring and Debugging Pthreads:

- Debuggers vary in their ability to handle Pthreads. The TotalView debugger is LC's recommended debugger for parallel programs. It is well suited for both monitoring and debugging threaded programs.
- An example screenshot from a TotalView session using a threaded code is shown below.
 1. Stack Trace Pane: Displays the call stack of routines that the selected thread is executing.
 2. Status Bars: Show status information for the selected thread and its associated process.
 3. Stack Frame Pane: Shows a selected thread's stack variables, registers, etc.
 4. Source Pane: Shows the source code for the selected thread.
 5. Root Window showing all threads
 6. Threads Pane: Shows threads associated with the selected process



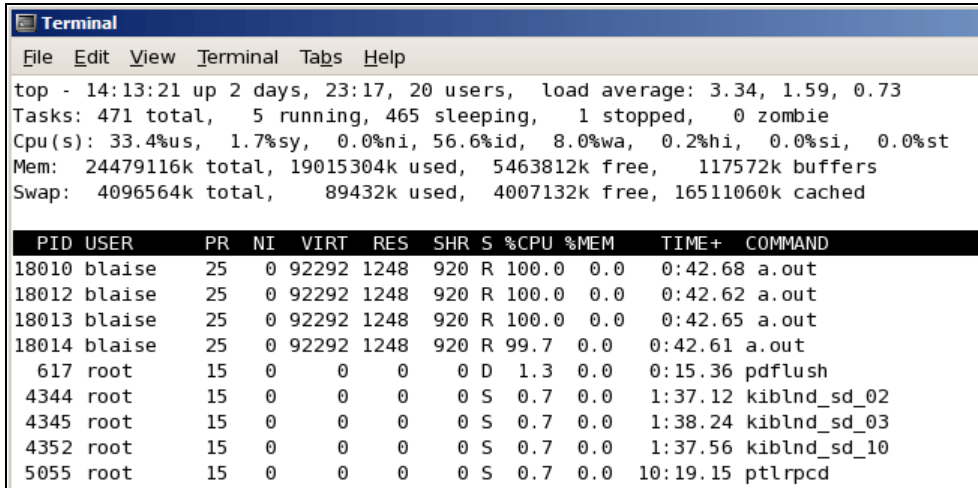
- See the [TotalView Debugger tutorial](#) for details.
- The Linux `ps` command provides several flags for viewing thread information. Some examples are shown below. See the [man page](#) for details.

```
% ps -Lf
UID      PID  PPID   LWP  C  NLWP  STIME  TTY          TIME CMD
blaise   22529 28240 22529  0    5  11:31 pts/53      00:00:00 a.out
blaise   22529 28240 22530  99    5  11:31 pts/53      00:01:24 a.out
blaise   22529 28240 22531  99    5  11:31 pts/53      00:01:24 a.out
blaise   22529 28240 22532  99    5  11:31 pts/53      00:01:24 a.out
blaise   22529 28240 22533  99    5  11:31 pts/53      00:01:24 a.out
```

```
% ps -T
  PID  SPID  TTY          TIME CMD
22529 22529 pts/53      00:00:00 a.out
22529 22530 pts/53      00:01:49 a.out
22529 22531 pts/53      00:01:49 a.out
22529 22532 pts/53      00:01:49 a.out
22529 22533 pts/53      00:01:49 a.out
```

```
% ps -Lm
  PID   LWP  TTY          TIME CMD
22529   -  pts/53      00:18:56 a.out
- 22529 -           00:00:00 -
- 22530 -           00:04:44 -
- 22531 -           00:04:44 -
- 22532 -           00:04:44 -
```

- LC's Linux clusters also provide the **top** command to monitor processes on a node. If used with the **-H** flag, the threads contained within a process will be visible. An example of the **top -H** command is shown below. The parent process is PID 18010 which spawned three threads, shown as PIDs 18012, 18013 and 18014.



```

top - 14:13:21 up 2 days, 23:17, 20 users, load average: 3.34, 1.59, 0.73
Tasks: 471 total, 5 running, 465 sleeping, 1 stopped, 0 zombie
Cpu(s): 33.4%us, 1.7%sy, 0.0%ni, 56.6%id, 8.0%wa, 0.2%hi, 0.0%si, 0.0%st
Mem: 24479116k total, 19015304k used, 5463812k free, 117572k buffers
Swap: 4096564k total, 89432k used, 4007132k free, 16511060k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 18010 blaise    25   0 92292 1248  920  R   100.0   0.0   0:42.68 a.out
 18012 blaise    25   0 92292 1248  920  R   100.0   0.0   0:42.62 a.out
 18013 blaise    25   0 92292 1248  920  R   100.0   0.0   0:42.65 a.out
 18014 blaise    25   0 92292 1248  920  R   99.7   0.0   0:42.61 a.out
    617 root       15   0     0     0     0  D    1.3   0.0   0:15.36 pdflush
   4344 root       15   0     0     0     0  S    0.7   0.0   1:37.12 kiblnd_sd_02
   4345 root       15   0     0     0     0  S    0.7   0.0   1:38.24 kiblnd_sd_03
   4352 root       15   0     0     0     0  S    0.7   0.0   1:37.56 kiblnd_sd_10
   5055 root       15   0     0     0     0  S    0.7   0.0  10:19.15 ptlrpcd
  
```

Performance Analysis Tools:

- There are a variety of performance analysis tools that can be used with threaded programs. Searching the web will turn up a wealth of information.
- At LC, the list of supported computing tools can be found at: computing.llnl.gov/code/content/software_tools.php.
- These tools vary significantly in their complexity, functionality and learning curve. Covering them in detail is beyond the scope of this tutorial.
- Some tools worth investigating, specifically for threaded codes, include:
 - Open|SpeedShop
 - TAU
 - HPCToolkit
 - PAPI
 - Intel VTune Amplifier
 - ThreadSpotter

LLNL Specific Information and Recommendations

This section describes details specific to Livermore Computing's systems.

Implementations:

- All LC production systems include a Pthreads implementation that follows draft 10 (final) of the POSIX standard. This is the preferred implementation.
- Implementations differ in the maximum number of threads that a process may create. They also differ in the default amount of thread stack space.

Compiling:

- LC maintains a number of compilers, and usually several different versions of each - see the [LC's Supported Compilers](#) web page.
- The compiler commands described in the [Compiling Threaded Programs](#) section apply to LC systems.

Mixing MPI with Pthreads:

- This is the primary motivation for using Pthreads at LC.
- Design:
 - Each MPI process typically creates and then manages N threads, where N makes the best use of the available cores/node.
 - Finding the best value for N will vary with the platform and your application's characteristics.
 - In general, there may be problems if multiple threads make MPI calls. The program may fail or behave unexpectedly. If MPI calls must be made from within a thread, they should be made only by one thread.
- Compiling:
 - Use the appropriate MPI compile command for the platform and language of choice
 - Be sure to include the required Pthreads flag as shown in the [Compiling Threaded Programs](#) section.
- An example code that uses both MPI and Pthreads is available below. The serial, threads-only, MPI-only and MPI-with-threads versions demonstrate one possible progression.
 - [Serial](#)
 - [Pthreads only](#)
 - [MPI only](#)
 - [MPI with pthreads](#)
 - [makefile](#)

Topics Not Covered

Several features of the Pthreads API are not covered in this tutorial. These are listed below. See the [Pthread Library Routines Reference](#) section for more information.

- Thread Scheduling
 - Implementations will differ on how threads are scheduled to run. In most cases, the default mechanism is adequate.
 - The Pthreads API provides routines to explicitly set thread scheduling policies and priorities which may override the default mechanisms.

- The API does not require implementations to support these features.
- Keys: Thread-Specific Data
 - As threads call and return from different routines, the local data on a thread's stack comes and goes.
 - To preserve stack data you can usually pass it as an argument from one routine to the next, or else store the data in a global variable associated with a thread.
 - Pthreads provides another, possibly more convenient and versatile, way of accomplishing this through **keys**.
- Mutex Protocol Attributes and Mutex Priority Management for the handling of "priority inversion" problems.
- Condition Variable Sharing - across processes
- Thread Cancellation
- Threads and Signals
- Synchronization constructs - barriers and locks

Pthread Exercise 2

Mutexes, Condition Variables and Hybrid MPI with Pthreads

Overview:

- Login to the LC workshop cluster, if you are not already logged in
- Mutexes: review and run the provided example codes
- Condition variables: review and run the provided example codes
- Hybrid MPI with Pthreads: review and run the provided example codes



[GO TO THE EXERCISE HERE](#)

This completes the tutorial.



Please complete the online evaluation form - unless you are doing the exercise, in which case please complete it at the end of the exercise.

Where would you like to go now?

- [Exercise](#)
- [Agenda](#)
- [Back to the top](#)

References and More Information

- Author: [Blaise Barney](#), Livermore Computing.
- POSIX Standard: www.unix.org/version3/ieee_std.html
- "Pthreads Programming". B. Nichols et al. O'Reilly and Associates.
- "Threads Primer". B. Lewis and D. Berg. Prentice Hall
- "Programming With POSIX Threads". D. Butenhof. Addison Wesley
- "Programming With Threads". S. Kleiman et al. Prentice Hall

Appendix A: Pthread Library Routines Reference

For convenience, an alphabetical list of Pthread routines, linked to their corresponding man page, is provided below.

[pthread_atfork](#)
[pthread_attr_destroy](#)
[pthread_attr_getdetachstate](#)
[pthread_attr_getguardsize](#)
[pthread_attr_getinheritsched](#)
[pthread_attr_getschedparam](#)
[pthread_attr_getschedpolicy](#)
[pthread_attr_getscope](#)
[pthread_attr_getstack](#)
[pthread_attr_getstackaddr](#)
[pthread_attr_getstacksize](#)
[pthread_attr_init](#)
[pthread_attr_setdetachstate](#)
[pthread_attr_setguardsize](#)
[pthread_attr_setinheritsched](#)
[pthread_attr_setschedparam](#)
[pthread_attr_setschedpolicy](#)
[pthread_attr_setscope](#)
[pthread_attr_setstack](#)
[pthread_attr_setstackaddr](#)
[pthread_attr_setstacksize](#)
[pthread_barrier_destroy](#)
[pthread_barrier_init](#)
[pthread_barrier_wait](#)
[pthread_barrierattr_destroy](#)
[pthread_barrierattr_getpshared](#)
[pthread_barrierattr_init](#)
[pthread_barrierattr_setpshared](#)
[pthread_cancel](#)
[pthread_cleanup_pop](#)
[pthread_cleanup_push](#)
[pthread_cond_broadcast](#)
[pthread_cond_destroy](#)
[pthread_cond_init](#)
[pthread_cond_signal](#)
[pthread_cond_timedwait](#)
[pthread_cond_wait](#)
[pthread_condattr_destroy](#)
[pthread_condattr_getclock](#)
[pthread_condattr_getpshared](#)
[pthread_condattr_init](#)
[pthread_condattr_setclock](#)
[pthread_condattr_setpshared](#)
[pthread_create](#)
[pthread_detach](#)
[pthread_equal](#)
[pthread_exit](#)
[pthread_getconcurrency](#)
[pthread_getcpuclockid](#)
[pthread_getschedparam](#)
[pthread_getspecific](#)
[pthread_join](#)
[pthread_key_create](#)
[pthread_key_delete](#)
[pthread_kill](#)
[pthread_mutex_destroy](#)
[pthread_mutex_getprioceiling](#)
[pthread_mutex_init](#)
[pthread_mutex_lock](#)
[pthread_mutex_setprioceiling](#)
[pthread_mutex_timedlock](#)
[pthread_mutex_trylock](#)
[pthread_mutex_unlock](#)
[pthread_mutexattr_destroy](#)
[pthread_mutexattr_getprioceiling](#)
[pthread_mutexattr_getprotocol](#)
[pthread_mutexattr_getpshared](#)
[pthread_mutexattr_gettype](#)
[pthread_mutexattr_init](#)
[pthread_mutexattr_setprioceiling](#)
[pthread_mutexattr_setprotocol](#)
[pthread_mutexattr_setpshared](#)
[pthread_mutexattr_settype](#)
[pthread_once](#)
[pthread_rwlock_destroy](#)
[pthread_rwlock_init](#)
[pthread_rwlock_rdlock](#)
[pthread_rwlock_timedrdlock](#)
[pthread_rwlock_timedwrlock](#)
[pthread_rwlock_tryrdlock](#)
[pthread_rwlock_trywrlock](#)
[pthread_rwlock_unlock](#)
[pthread_rwlock_wrlock](#)
[pthread_rwlockattr_destroy](#)
[pthread_rwlockattr_getpshared](#)
[pthread_rwlockattr_init](#)
[pthread_rwlockattr_setpshared](#)
[pthread_self](#)
[pthread_setcancelstate](#)
[pthread_setcanceltype](#)
[pthread_setconcurrency](#)
[pthread_setschedparam](#)
[pthread_setschedprio](#)
[pthread_setspecific](#)
[pthread_sigmask](#)
[pthread_spin_destroy](#)
[pthread_spin_init](#)
[pthread_spin_lock](#)
[pthread_spin_trylock](#)
[pthread_spin_unlock](#)
[pthread_testcancel](#)

<https://computing.llnl.gov/tutorials/pthreads/>
Last Modified: 03/07/2017 23:54:00 blaiseb@llnl.gov
UCRL-MI-133316

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.